Two-wheel self-balancing robot that maintains an upright position by acting as an inverted pendulum.

In Partial Fulfilment of the Summer Internship Program Requirements

Submitted By

Varun Patil

Himanshu Pawar

Ashwin Pardeshi

Under The Guidance Of

Kirit Nandaniya

Submitted To

KPIT APEX LAB

Academic Year 2025-2026

ACKNOWLEDGEMENT

The successful completion of this project titled “Autonomous Two-Wheel Self-Balancing Robot Using PID Control” would not have been possible without the guidance, support, and encouragement received from various individuals and organizations throughout the development process.

First and foremost, sincere gratitude is expressed to the Department of KPIT APEX Lab for providing the opportunity to work on this research-based project. The project provided valuable exposure to the fields of modern control theory, sensor fusion, embedded systems, and real-time algorithmic implementation.

Special thanks are extended to our mentor, Kirit Sir, for providing the project problem statement, continuous technical guidance, valuable suggestions, and the regular project review meetings conducted throughout the development duration. His mentorship played an important role in understanding the complex dynamics of the inverted pendulum, solving technical challenges related to PID tuning, and improving the hardware implementation approach during each phase of development.

Gratitude is also expressed to the faculty members, lab coordinators, teammates, and colleagues who provided support during hardware assembly, circuit troubleshooting, software debugging, testing, and documentation activities throughout the project period.

Special appreciation is also given to the developers and contributors of open-source technologies such as the Arduino IDE, AccelStepper library, Wire library, and MPU6050 open-source drivers, which made the implementation of this project possible using accessible and affordable embedded hardware platforms.

Finally, heartfelt thanks are expressed to family members for their continuous encouragement, patience, and moral support during the successful completion of the project and its documentation.

ABSTRACT

The "Autonomous Two-Wheel Self-Balancing Robot" project focuses on developing a robust, real-time control system capable of stabilizing an inherently unstable inverted pendulum mechanism. The primary objective is to maintain a continuous upright position and execute controlled linear (forward and backward) and rotational (yaw) movements without losing balance. This is achieved by employing a highly responsive, closed-loop Proportional-Integral-Derivative (PID) control algorithm operating efficiently at a precise 100Hz frequency control loop.

The proposed hardware architecture utilizes an Arduino Mega 2560 as the primary embedded processing unit, chosen specifically for its superior I/O capabilities and the processing headroom required to handle complex floating-point mathematics alongside real-time sensor polling. An MPU6050 6-Degree-of-Freedom (DOF) Inertial Measurement Unit (IMU) is integrated via I2C protocol to continuously acquire real-time spatial orientation and angular velocity. To achieve highly precise actuation and eliminate the mechanical backlash commonly found in standard DC gear motors, two NEMA 17 Stepper Motors driven by A4988 motor driver modules configured for 1/16 microstepping are deployed. A mathematical Complementary Filter algorithm fuses the raw accelerometer and gyroscope data, successfully mitigating high-frequency sensor noise, mechanical vibrations, and low-frequency integration drift to provide a highly accurate and stable tilt angle calculation.

The project encompasses the complete lifecycle of embedded systems development, including the mathematical modeling of the inverted pendulum, dynamic hardware assembly, automated sensor offset calibration, and embedded software programming in C++. The developed prototype successfully demonstrates autonomous self-balancing, stable recovery from external physical disturbances, and controlled differential steering. Furthermore, a non-blocking serial communication interface was implemented to issue dynamic motion commands and monitor critical system states in real time.

This project establishes a strong practical foundation in modern control theory, sensor fusion, and embedded robotics. Future improvements such as wireless Bluetooth remote control, autonomous obstacle avoidance using ultrasonic sensors, and advanced LQR (Linear Quadratic Regulator) control algorithms can be seamlessly integrated into this existing, highly capable hardware architecture to create a comprehensive autonomous vehicle.

Table of Content

Acknowledgement....2
Abstract....3
Table of Contents....4
List of Symbols....6
List of Abbreviations....7

CHAPTER 1: INTRODUCTION....9
1.1 Project Overview....9
1.2 Objectives of the Project....9
1.3 Concept of the Inverted Pendulum....9

CHAPTER 2: LITERATURE REVIEW....11
2.1 Study of Self-Balancing Systems....11
2.2 Review of Filtering Algorithms (Complementary vs. Kalman)....11

CHAPTER 3: HARDWARE ARCHITECTURE & COMPONENTS....13
3.1 Introduction....13
3.2 Arduino Mega 2560....13
3.2.1 Key Specifications....13
3.2.2 Why Arduino Mega for Self-Balancing Robot....14
3.2.3 Typical Pin Allocation....15
3.2.4 Datasheet Reference....15
3.3 MPU6050 IMU....15
3.3.1 Gyroscope Specifications....16
3.3.2 Accelerometer Specifications....17
3.3.3 Sensor Fusion....17
3.3.4 Power Specifications....17
3.3.5 Mechanical Mounting....18
3.4 NEMA 17 Stepper Motor....18
3.4.1 Electrical Specifications....18
3.4.2 Operating Principle....19
3.5 A4988 Stepper Driver....19
3.5.1 Key Electrical Specifications....20
3.5.2 Control Logic Pins....21
3.5.3 Current Limiting....21

3.6 Power Supply and Chassis Design 22

CHAPTER 4: SYSTEM WIRING & PINOUT CONFIGURATION 23
4.1 Circuit Diagram Overview 23
4.2 Arduino Pin Mapping 24
4.3 A4988 Driver Connections 24
4.4 MPU6050 Connections 26

CHAPTER 5: CONTROL SYSTEM & SOFTWARE IMPLEMENTATION 27
5.1 Introduction to PID Controller 27
5.2 Mathematical Model of PID Controller 28
5.2.1 Continuous-Time PID Equation 28
5.2.2 Discrete-Time PID Equation 29
5.3 Proportional Control (P) 30
5.3.1 Theory and Working Principle 30
5.3.2 Mathematical Equation 30
5.3.3 Effect on Self-Balancing Robot Stability 30
5.3.4 Advantages and Limitations 31
5.4 Integral Control(I) 31
5.4.1 Theory and Working Principle 31
5.4.2 Mathematical Equation 32
5.4.3 Steady-State Error 32
5.4.4 Advantages and Limitations 33
5.5 Derivative Control (D) 33
5.5.1 Theory and Working Principle 33
5.5.2 Mathematical Equation and Derivative-on-Measurement Formulation 34
5.5.3 Derivative Low-Pass Filter 34
5.5.4 Advantages and Limitations 35
5.6 Combined PID Controller 35
5.6.1 Interaction Between P, I, and D Terms 35
5.7 PID Control Block Diagram 36
5.8 PID Tuning 37
5.8.1 Manual Tuning 37
5.8.2 Trial-and-Error Tuning 37
5.8.3 Ziegler–Nichols Tuning Method 37
5.8.4 Practical Tuning Procedure for the Self-Balancing Robot 38
5.9 PID Implementation in the Self-Balancing Robot 39
5.9.1 Angle Acquisition from MPU6050 39
5.9.2 Error Computation 39
5.9.3 PID Computation and Motor Speed Output 40

5.9.4 Velocity Damping Augmentation....40
5.9.5 Final Tuned PID Parameter Values....41
5.10 Advantages and Limitations of PID Control....41

CHAPTER 6: EXPERIMENTAL RESULTS & ANALYSIS....43
6.1 Main Loop Architecture....43
6.1.1 Tier 1: High-Frequency Motor Pulse Execution....43
6.1.2 Tier 2: 100 Hz Control Loop (10 ms Gate)....44
6.2 Function Descriptions....46
6.2.1 initMPU()....46
6.2.2 calibrate()....47
6.2.3 readSensor()....47
6.2.4 complementaryFilter()....48
6.2.5 pidUpdate()....48
6.2.6 setMotorSpeed()....49
6.2.7 Fall Detection State Machine....50
6.3 Tunable System Parameters....51
6.4 Summary....53

CHAPTER 7: FUTURE IMPROVEMENTS....54
7.1 Wireless Teleoperation....54
7.2 Autonomous Obstacle Avoidance....54
7.3 Advanced Control Algorithms....54
7.4 Adaptive and Auto-Tuning PID....54
7.5 IMU Sensor Fusion Upgrades....55
7.6 Power Efficiency and Management....55
7.7 Data Logging....55

CHAPTER 8: CONCLUSION....56

References....57

List of Symbols

Symbol	Description
u(t) / u[n]	Control Output: The control signal generated by the PID controller and applied to the motors.
e(t) / e[n]	Error Signal: The difference between the reference setpoint and the measured process variable.
Kp	Proportional Gain: A tuning constant that scales the proportional control action.
Ki	Integral Gain: A tuning constant that scales the integral control action.
Kd	Derivative Gain: A tuning constant that scales the derivative control action.
$\int e(t)dt$	Integral of Error: The cumulative sum of all past error values over time.
$de(t)/dt$	Derivative of Error: The instantaneous rate of change of the error signal.
Δt / dt	Fixed sampling interval in seconds / Elapsed time.
α	Filter coefficient: Used for the low-pass derivative filter or complementary filter.
θ_setpoint	Target balance angle.
θ_measured(t)	Current filtered tilt angle estimate provided by the Complementary Filter.
g	Unit of gravitational acceleration.

List of Abbreviations

Abbreviation	Full Form
PID	Proportional-Integral-Derivative
DOF	Degree of Freedom
IMU	Inertial Measurement Unit
I2C	Inter-Integrated Circuit
LQR	Linear Quadratic Regulator
CoG	Center of Gravity
MEMS	Micro-Electro-Mechanical Systems
LTI	Linear Time-Invariant
IR	Impulse Response
DoM	Derivative of Measurement
ADC	Analog-to-Digital Converter
OCR	Output Data Rate
PWM	Pulse Width Modulation
UART	Universal Asynchronous Receiver-Transmitter
GPIO	General Purpose Input/Output
DMP	Digital Motion Processor

CHAPTER 1 : INTRODUCTION

1.1 Project Overview

The development of autonomous robotics has seen significant advancements over the past decade, heavily integrating mechanical design, embedded electronics, and complex software algorithms. The two-wheel self-balancing robot represents a quintessential problem in modern control theory. Unlike traditional four-wheeled vehicles that are statically stable, a two-wheeled robot is dynamically stable but statically unstable. It requires continuous micro-adjustments to its wheel positioning to counteract the force of gravity.

This project aims to engineer a highly responsive, autonomous self-balancing robot from the ground up. The system leverages an Arduino Mega 2560 microcontroller to interface with an MPU6050 Inertial Measurement Unit (IMU). By reading raw acceleration and gyroscopic data, the system calculates the robot's real-time pitch angle. A Proportional-Integral-Derivative (PID) controller uses this angle to compute the necessary restorative force, which is then translated into precise step commands fed to two NEMA 17 stepper motors. The combination of stepper motors and a high-frequency (100Hz) control loop provides the rapid torque response required to maintain equilibrium and perform dynamic linear and rotational translations.

1.2 Objectives of the Project

The primary goal of this project is to successfully implement a closed-loop control system capable of balancing an inverted pendulum model. The specific objectives include:

- **Hardware Integration:** To design and construct a structurally rigid two-wheel chassis accommodating the Arduino Mega, stepper motors, motor drivers, and power supply.
- **Sensor Data Fusion:** To develop a robust sensor fusion algorithm (Complementary Filter) that mathematically combines accelerometer and gyroscope data to accurately determine the robot's tilt angle, effectively eliminating signal noise and gyroscopic drift.
- **Algorithm Development:** To implement and tune a software-based PID control algorithm operating at a strict 100Hz loop frequency to compute corrective motor velocities.
- **Precise Actuation:** To interface and drive NEMA 17 stepper motors using A4988 drivers with 1/16 microstepping, ensuring smooth, high-resolution torque delivery without mechanical backlash.
- **Dynamic Translation:** To advance beyond static balancing by dynamically adjusting the pitch setpoint, allowing the robot to execute forward, backward, and rotational (yaw) movements while maintaining stability via a non-blocking serial command interface.

1.3 Concept of the Inverted Pendulum

The physics behind the self-balancing robot is fundamentally based on the "Inverted Pendulum" model. In a standard pendulum, the center of gravity (CoG) is located below the pivot point, resulting in a naturally stable system that eventually comes to rest at the lowest point due to gravity.

Conversely, an inverted pendulum has its center of gravity located *above* its pivot point (the axis of the wheels). This makes the system inherently unstable; any slight deviation from the absolute vertical axis will cause the force of gravity to accelerate the mass downward, leading to a fall.

To prevent this fall, the pivot point must be continuously moved in the direction of the tilt. If the robot leans forward, the control system must detect this angular displacement and rapidly accelerate the wheels forward to force the pivot point back under the center of gravity. This continuous, real-time cycle of sensing the tilt, calculating the required restorative acceleration, and driving the wheels forms the foundation of the robot's active balancing system.

CHAPTER 2: LITERATURE REVIEW

2.1 Study of Self-Balancing Systems

The concept of actively balanced two-wheeled vehicles gained widespread commercial and academic attention following the invention of the Segway. Early iterations of these systems relied heavily on expensive, industrial-grade gyroscopes and complex, power-hungry processing units to maintain equilibrium. However, the advent of Micro-Electro-Mechanical Systems (MEMS) technology has revolutionized the field. Sensors like the MPU6050 have drastically reduced the physical footprint and cost of Inertial Measurement Units (IMUs), making it possible to implement sophisticated balancing algorithms on affordable embedded platforms.

In modern control engineering, self-balancing systems generally employ one of two primary control strategies: Proportional-Integral-Derivative (PID) control or Linear Quadratic Regulator (LQR) control. LQR is a state-space approach that provides optimal multi-variable control, calculating gains based on a predefined cost function. While mathematically superior for highly complex, multi-axis systems, LQR requires a highly accurate mathematical model of the robot's physical dynamics (including mass distribution, motor torque constants, and wheel inertia), which is often difficult to derive perfectly for custom-built hardware.

Conversely, PID control remains the ubiquitous industry standard for single-axis balancing systems. Its popularity stems from its robust performance, intuitive tuning methodology, and minimal computational overhead. By treating the system as a black box and tuning the P, I, and D gains based on empirical observation (such as reacting to oscillations or steady-state drift), a PID controller can effectively stabilize the robot without requiring a perfect mathematical model of the physical hardware. This makes it highly suitable for 8-bit microcontrollers like the ATmega2560 used in this project.

2.2 Review of Filtering Algorithms (Complementary vs. Kalman)

Accurate and instantaneous tilt angle calculation is the most critical prerequisite for an inverted pendulum control system. A microcontroller cannot balance the robot if it does not know the exact angle at which it is currently leaning. The MPU6050 provides two sources of orientation data: the accelerometer and the gyroscope.

Raw accelerometer data is highly susceptible to high-frequency mechanical noise, such as chassis vibrations and the physical stepping actions of the motors. While it provides an accurate reference

to Earth's gravity over a long period, it is useless for instantaneous angle measurement. Conversely, the gyroscope provides excellent instantaneous angular velocity, but when integrated over time to find the angle, it suffers from "gyroscopic drift" (low-frequency noise) due to tiny cumulative measurement errors. Therefore, sensor fusion algorithms are mandatory to extract a clean signal.

The **Kalman Filter** is widely considered the gold standard for sensor fusion. It is a recursive algorithm that mathematically estimates the true state of a system by minimizing the mean of the squared error. While it handles both linear and non-linear noise exceptionally well, the Kalman Filter relies on extensive matrix multiplication. This computational complexity can severely bottleneck the processing capabilities of an 8-bit microcontroller, making it difficult to sustain the strict, high-frequency (100Hz) control loop required for reactive balancing.

To overcome this computational limitation, the **Complementary Filter** is utilized in this project. The Complementary Filter offers an elegant, mathematically lightweight alternative to the Kalman Filter. It operates on the principle of applying a digital low-pass filter to the noisy accelerometer data (to ignore short-term vibrations) and a high-pass filter to the integrating gyroscope data (to ignore long-term drift).

The formula is generally expressed as: `Angle = α * (Angle + GyroData * dt) + (1 - α) * (AccelerometerData)`

Where `α` (Alpha) is typically a high value such as `0.98`. In the context of this project, the Complementary Filter provides a highly stable and responsive angle estimation that rivals the Kalman Filter in practical performance, while requiring only a fraction of the CPU time. This computational efficiency is what enables the Arduino Mega to read the sensors, filter the data, compute the PID output, and pulse the stepper motors all within a strict 10-millisecond window.

CHAPTER 3:- HARDWARE ARCHITECTURE & COMPONENTS

3.1 Introduction

The physical stability of a self-balancing robot is directly proportional to the quality and responsiveness of its hardware. The hardware architecture selected for this project prioritizes real-time processing speed, highly accurate spatial sensing, and zero-backlash mechanical actuation.

3.2 Arduino Mega 2560 (ATmega2560)

The Arduino Mega 2560 serves as the primary "brain" of the robot. Powered by the ATmega2560 8-bit microcontroller running at 16 MHz, it handles all floating-point mathematics, I2C sensor communication, and motor pulse generation. While an Arduino UNO is technically capable of balancing, the Mega 2560 was chosen to provide sufficient SRAM and flash memory overhead for future expansions, such as Bluetooth teleoperation and autonomous pathfinding.

3.2.1 Key Specifications

Parameter	Value
Microcontroller	ATmega2560
Operating Voltage	5V
Input Voltage (recommended)	7–12V
Input Voltage (limit)	6–20V

Digital I/O Pins	54 (15 provide PWM output)
Analog Input Pins	16
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	256 KB (8 KB used by bootloader)
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz
UART (Serial) Ports	4 (Serial, Serial1, Serial2, Serial3)
I2C (SDA/SCL)	Pin 20 (SDA), Pin 21 (SCL)
SPI Pins	50 (MISO), 51 (MOSI), 52 (SCK), 53 (SS)
External Interrupt Pins	6 (Pins 2, 3, 18, 19, 20, 21)
USB	1
Power Jack	Yes (barrel jack)
Dimensions	101.52 mm x 53.3 mm

3.2.2 Why Arduino Mega for a Self-Balancing Robot

Requirement	How Mega Helps
IMU (MPU6050) via I2C	Dedicated I2C pins (20, 21) free up other digital pins
Motor driver (e.g., L298N/TB6612)	Needs 4–6 digital/PWM pins per motor — Mega's 15 PWM pins give headroom
Quadrature encoders (optional)	6 external interrupt pins allow attaching interrupts to 2 motors' encoder channels simultaneously
Bluetooth/Wi-Fi module (HC-05/ESP)	Extra hardware serial ports (Serial1–3) avoid conflicts with USB debugging on Serial
PID tuning via Serial Monitor	Can use one UART for debug prints while another handles wireless control
Future expansion (ultrasonic, LCD, joystick)	Abundant digital/analog pins reduce need for multiplexing

3.2.3 Typical Pin Allocation Example

Function	Mega Pin(s)
MPU6050 SDA	20
MPU6050 SCL	21
Motor A PWM	5
Motor A DIR1/DIR2	6, 7

Motor B PWM	9
Motor B DIR1/DIR2	10, 11
Encoder A (optional)	2 (interrupt)
Encoder B (optional)	3 (interrupt)
Bluetooth (HC-05) TX/RX	Serial1 (TX1=18, RX1=19)
Debug Serial	Serial (USB, pins 0/1)

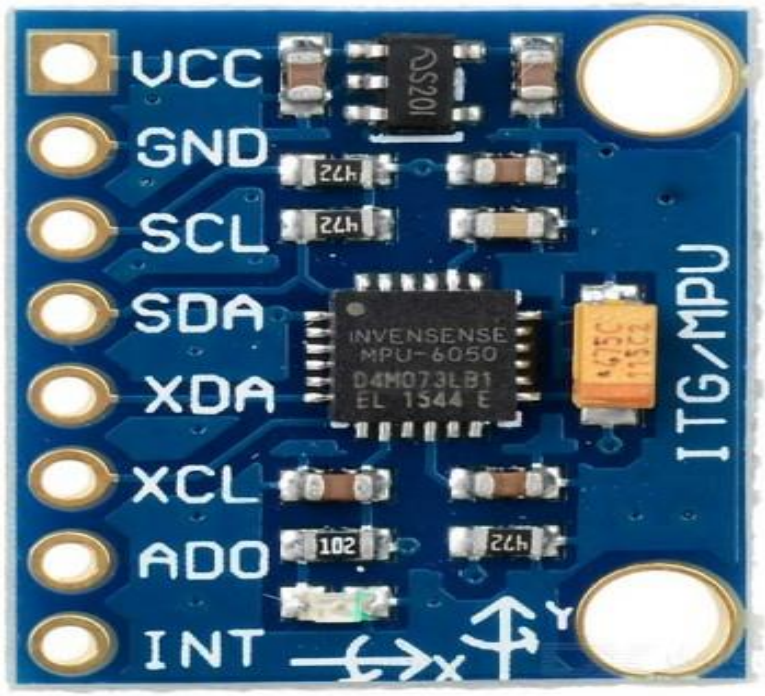
3.2.4 Datasheet References

https://docs.arduino.cc/resources/datasheets/A000067-datasheet

https://docs.arduino.cc/hardware/mega-2560/

3.3 MPU6050 (6-DOF IMU)

The MPU6050 is a Micro-Electro-Mechanical Systems (MEMS) sensor combining a 3-axis accelerometer and a 3-axis gyroscope on a single silicon die. It communicates with the Arduino via the I2C protocol at a high speed of 400kHz. In this project, the Y and Z axes of the accelerometer are used to calculate the static tilt angle relative to gravity, while the X-axis of the gyroscope provides the fast-acting rotational velocity required to detect sudden falls.



3.3.1 Gyroscope Specifications (relevant for angular rate / pitch tracking)

Parameter	Value (from datasheet)
Full-scale range	User-programmable ±250, ±500, ±1000, or ±2000°/sec
ADC resolution	16-bit ADC per axis
Sensitivity scale factor	131 LSB/(°/s) at ±250°/s; 65.5 at ±500°/s; 32.8 at ±1000°/s; 16.4 at ±2000°/s
Output Data Rate (ODR)	Programmable from 4 Hz up to 8,000 Hz
Low-pass filter range	Digitally programmable cutoff from 5 Hz to 256 Hz
Operating current	3.6 mA gyroscope-only operating current
Start-up settling time	~30 ms for zero-rate output to settle to within ±1°/s of final value

Balancing robot relevance: The **Y-axis gyroscope** (pitch rate) is the primary signal used to track how fast the robot is tilting forward/backward. A narrower full-scale range (e.g. ±250°/s) gives higher resolution for the small, fast corrections typical in balancing.

3.3.2 Accelerometer Specifications (relevant for absolute tilt angle)

Parameter	Value (from datasheet)
Full-scale range	Programmable ±2g, ±4g, ±8g, or ±16g
ADC resolution	16-bit, two's complement output
Sensitivity scale factor	16,384 LSB/g at ±2g; 8,192 at ±4g; 4,096 at ±8g; 2,048 at ±16g
Output Data Rate	Programmable from 4 Hz to 1,000 Hz
Operating current	500 µA in accelerometer-only mode
Zero-G reference	On a flat surface, the device reads 0g on X and Y axes and +1g on the Z axis

Balancing robot relevance: The **X and Z axis accelerometers** give the gravity vector, from which tilt angle is computed (`atan2(accel_x, accel_z)`). Since accelerometers are noisy under vibration (motor-induced), this is fused with gyro data — hence the need for a complementary/Kalman filter.

3.2.3 Why Sensor Fusion Is Needed (datasheet-supported reasoning)

- Gyroscope alone **drifts** over time — datasheet notes zero-rate output tolerance of ±20°/s and variation over temperature of ±20°/s, meaning pure integration of gyro data accumulates error.
- Accelerometer alone is **noisy under vibration** — the datasheet lists a noise power spectral density of 400 µg/√Hz at 10 Hz with ODR = 1kHz, and robot motors/vibration

inject additional noise on top of this. Tis is precisely why balancing robots fuse both signals (Kalman/complementary filter) rather than relying on either sensor alone.

3.2.4 Power Specifications

Parameter	Value
VDD operating range	2.375V to 3.46V
VLOGIC (I2C logic level pin, MPU-6050 only)	1.71V to VDD
Normal operating current (gyro+accel+DMP)	3.9 mA
Full-chip idle current	5 µA

3.2.5 Mechanical / Mounting Notes

- Routing active signal traces under the gyro package is prohibited, since coupled signals can harmonically interfere with the MEMS gyro elements (drive frequencies ~27-33 kHz per axis).
- Large components such as buttons or connectors should be kept at least 6 mm away from the MEMS gyro to avoid mechanical/thermal stress affecting readings.

3.4 NEMA 17 Stepper Motors

To physically balance the robot, NEMA 17 stepper motors were selected over traditional DC gear motors. DC gear motors suffer from mechanical "backlash"—a slight looseness in the gears that causes a delay when the motor abruptly changes direction. Since a balancing robot must oscillate back and forth rapidly to stay upright, any backlash leads to severe instability. Stepper motors eliminate this issue entirely, providing direct-drive torque and precise step-by-step angular positioning.

3.4.1 Electrical Specifications (from datasheet)

Parameter	Value
Motor type	4-wire bipolar stepper
Step angle	1.8° per step → 200 steps per revolution (full step)
Phase voltage	2.6 Vdc
Phase current	1.7 A per phase
Resistance per phase	1.5Ω ±10%

3.4.2Operating Principle

A NEMA17 is a **hybrid stepping motor** — it combines the working principles of a permanent-magnet stepper and a variable-reluctance stepper. Internally:

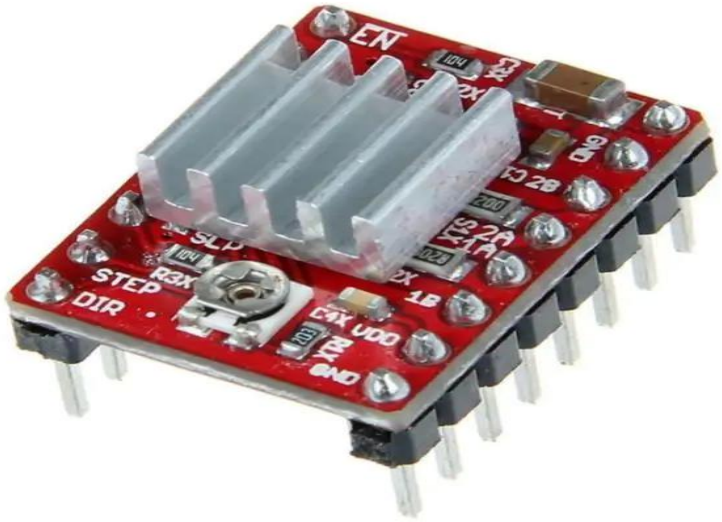
- The **rotor** contains permanent magnets.
- The **stator** has multiple coil windings (organized into 2 phases for a bipolar motor, or 2 phases with center-taps for a unipolar/6-wire motor).
- The motor has six wires connected to two split windings in the unipolar version — Black, Yellow, Green form the first winding while Red, White, Blue form the second. In the simpler 4-wire **bipolar** version (like the 17HS4401), each winding has just 2 ends (no center tap) — giving stronger torque per amp but requiring an H-bridge driver capable of reversing current direction in each coil.

Step sequence: A driver IC (A4988, DRV8825, TB6600, TMC2208, etc.) energizes the two coils in a specific switching pattern. As current alternates through Phase A and Phase B in a 4-step (or finer microstepped) sequence, the rotor's magnetic poles are pulled to align with the stator's currently-energized poles — rotating it by exactly one step angle (1.8°) each time. Repeating this

sequence 200 times completes one full revolution. The stator pole order in the motor follows A, B, A', B', and a driver IC is required to handle the current needed by these coils.

3.5 A4988 Stepper Motor Drivers

Stepper motors cannot be driven directly by microcontroller pins due to high current requirements. The A4988 motor driver modules are used as current amplifiers and translators. In this system, the A4988 drivers are configured for **1/16 Microstepping** by pulling the MS1, MS2, and MS3 pins HIGH. This translates the standard 200 steps-per-revolution of the NEMA 17 motor into an ultra-smooth 3200 steps-per-revolution, virtually eliminating mechanical vibrations that could otherwise interfere with the MPU6050 sensor.



3.5.1 Key Electrical Specifications

Parameter	Value (from datasheet)
Load supply voltage (VBB)	8 V to 35 V (operating) Dasenic
Logic supply voltage (VDD)	3 V to 5.5 V Dasenic — directly compatible with both 3.3V (STM32) and 5V (Arduino) logic
Max output current per phase	±2 A absolute maximum Dasenic
Continuous output on-resistance	320–430 mΩ for both source and sink drivers at 1.5A Dasenic

Overcurrent protection threshold		2.1 A typical Dasenic	
Thermal shutdown temperature		165°C, with 15°C hysteresis Dasenic	
Package		28-contact QFN, 5mm × 5mm × 0.90mm, with exposed thermal pad Dasenic	
Step modes		Five selectable: full, 1/2, 1/4, 1/8, and 1/16 step	

3.5.2 How It Controls the Motor (Control Logic Pins)

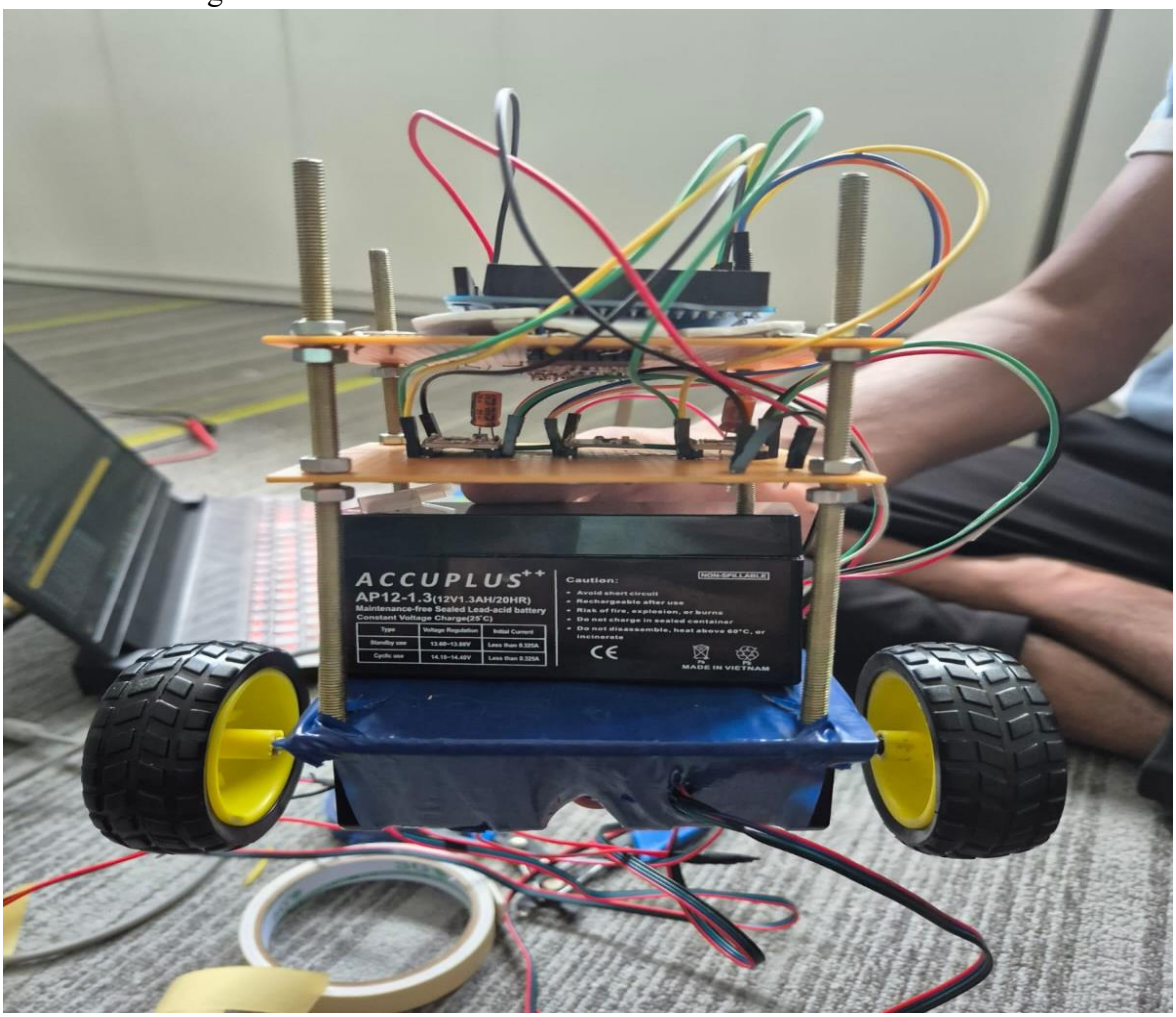
Pin	Function
STEP	A low-to-high transition on STEP sequences the translator and advances the motor by one increment
DIR	Determines the direction of rotation; changes take effect on the next STEP rising edge
MS1, MS2, MS3	Set the microstep resolution per the truth table — e.g. all LOW = full step (2-phase mode), all HIGH = 1/16 step
ENABLE	Turns on/off all FET outputs — logic high disables outputs, logic low enables them as required
SLEEP	A logic low puts the device into Sleep mode to minimize power; a 1 ms delay is needed after waking before issuing a Step command
RESET	Sets the translator to a predefined Home state and turns off all FET outputs
VREF	Sets max current limit via the formula below

3.5.3 Current Limiting

The maximum current limit, ITripMAX, is set by ITripMAX = VREF / (8 × RS), where RS is the sense resistor value and VREF is the input voltage on the REF pin. This is why every A4988 tutorial tells you to turn the onboard potentiometer while measuring VREF — you're directly setting how much current your NEMA17 will actually receive, protecting it from overcurrent/overheating.

3.6 Power Supply and Chassis Design

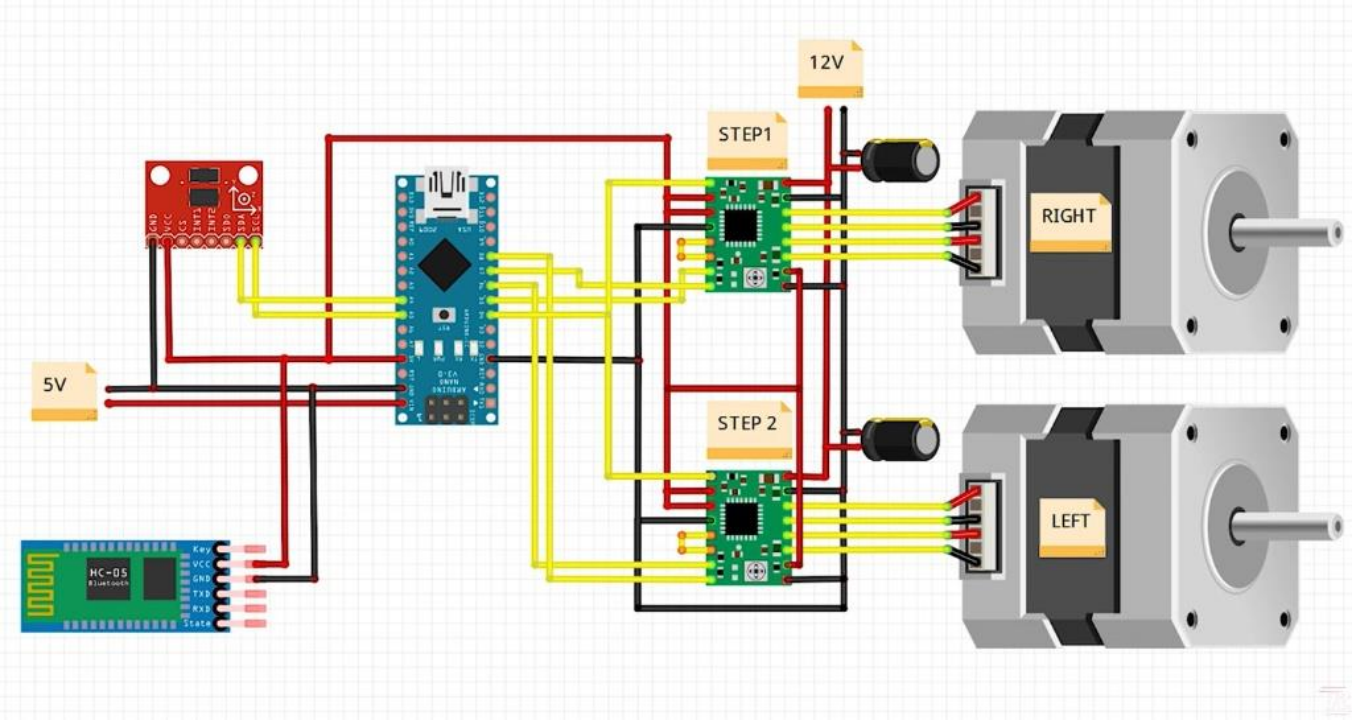
The system relies on a continuous and robust power supply, typically provided by high-discharge Li-Po or Lithium-Ion batteries, to handle the sudden current spikes drawn by the stepper motors when aggressively counteracting a fall. The chassis is structurally designed to keep the center of gravity optimally positioned and rigidly secures the sensor to prevent false readings caused by structural flexing.



CHAPTER 4 :- SYSTEM WIRING & PINOUT CONFIGURATION

4.1 Circuit Diagram Overview

The wiring architecture of the robot integrates the processing unit, sensor, and actuators. The MPU6050 is connected via the dedicated I2C pins, while the A4988 drivers utilize standard digital GPIO pins to receive rapid Step and Direction pulses from the Arduino. A centralized `ENABLE` pin controls the power state of both drivers simultaneously, acting as a critical hardware safety cutoff if the robot falls beyond its recovery threshold.



4.2 Pin Mapping Table Of Arduino

The following table reflects the exact hardware connections defined in the firmware (`#define` constants).

Component	Arduino Mega Pin	Description / Function
MPU6050 (IMU)	Pin 20 (SDA)	I2C Data Line
MPU6050 (IMU)	Pin 21 (SCL)	I2C Clock Line
A4988 (Common)	Digital Pin 4	Driver ENABLE (Active LOW; HIGH = Drivers OFF)
A4988 (Left)	Digital Pin 5	STEP Pulse Input (Motor 1)
A4988 (Left)	Digital Pin 7	DIR Direction Control (Motor 1)
A4988 (Right)	Digital Pin 6	STEP Pulse Input (Motor 2)
A4988 (Right)	Digital Pin 8	DIR Direction Control (Motor 2)

4.3 Pin Mapping Table Of Motor Driver A4988

A4988 Pin	Function	Connects To
VMOT	Motor power ground	External power supply GND
GND (motor side)	Motor power ground	External power supply GND
VDD	Logic power supply (+)	Microcontroller 5V (or 3.3V)
GND (logic side)	Logic ground	Microcontroller GND
1A	Motor coil A terminal 1	Stepper motor Coil A+

1B	Motor coil A terminal 2	Stepper motor Coil A−
2A	Motor coil B terminal 1	Stepper motor Coil B+
2B	Motor coil B terminal 2	Stepper motor Coil B−
STEP	Step pulse input	Microcontroller digital pin (e.g., D3)
DIR	Direction control input	Microcontroller digital pin (e.g., D4)
ENABLE	Enable/disable driver outputs (active LOW)	Microcontroller digital pin or GND (to keep always enabled)
MS1	Microstep select 1	Microcontroller pin or GND/VDD (set resolution)
MS2	Microstep select 2	Microcontroller pin or GND/VDD
MS3	Microstep select 3	Microcontroller pin or GND/VDD
RESET	Resets driver (active LOW)	Tie to SLEEP pin or VDD
SLEEP	Sleep mode (active LOW)	Tie to RESET pin or VDD (to keep awake)

4.4 Pin Mapping Table Of MPU 6050

MPU6050 Pin	Function	Connects To
VCC	Power (3.3V or 5V*)	3V3
GND	Ground	GND
SCL	I2C Clock	SCL Pin of Ardiuno Mega
SDA	I2C Data	
XDA	Aux I2C Data (master mode)	Not connected
XCL	I2C Address Select	Not connected
AD0	I2C Address Select	GND (or VCC)
INT	Data Ready Interrupt	Any GPIO (optional)

CHAPTER :- 5 CONTROL SYSTEM & SOFTWARE IMPLEMENTATION

5.1 Introduction to PID Controller

The Proportional-Integral-Derivative (PID) controller is the most widely deployed feedback control algorithm in industrial and embedded control applications. Surveys of industrial practice

consistently indicate that more than 90 percent of regulatory control loops in process industries employ some variant of PID control, a prevalence attributable to the controller's conceptual simplicity, ease of implementation, intuitive tuning methodology, and robust performance across a broad class of systems. In the context of the self-balancing robot, the PID controller serves as the computational core of the balance control loop, translating the measured tilt angle into an appropriate motor speed command at every control cycle.

The PID controller operates by computing an error signal, defined as the algebraic difference between the desired setpoint and the measured process variable, and generating a control output that is a linear combination of three mathematically distinct terms. The proportional term produces a control action proportional to the instantaneous magnitude of the error; the integral term accumulates the error over time and generates a control action proportional to the total accumulated error; and the derivative term responds to the rate of change of the error and generates a control action proportional to the speed at which the error is changing. Each of these three terms addresses a fundamentally different aspect of the system's dynamic behaviour, and their combined effect, when the gains are appropriately tuned, produces a controller capable of achieving rapid response, zero steady-state error, and adequate damping simultaneously.

The PID controller is classified as a linear time-invariant (LTI) controller when the plant is assumed to be linear. In practice, the dynamics of the self-balancing robot involve nonlinear elements including actuator saturation, friction, and the nonlinear relationship between tilt angle and gravitational torque. Nevertheless, the PID controller provides satisfactory performance in the region of operation near the balance point, where the system behaves approximately linearly, and is therefore well-suited to this application.

5.2 Mathematical Model of PID Controller

5.2.1 Continuous-Time PID Equation

The continuous-time formulation of the PID control law is expressed in terms of the error signal e(t) and its integral and derivative as follows:

$$u(t) = Kp \cdot e(t) + Ki \cdot \int e(t)dt + Kd \cdot [de(t) / dt] \qquad (5.1)$$

where the variables and parameters are defined as follows:

Symbol	Parameter	Definition
u(t)	Control Output	The control signal generated by the PID controller and applied to the stepper motors as a speed command, expressed in steps per second.
e(t)	Error Signal	The difference between the reference setpoint (balance angle) and the measured process variable (actual tilt angle): e(t) = Setpoint − Measurement.
Kp	Proportional Gain	A dimensionless tuning constant that scales the proportional control action. Higher values produce a stronger restoring force per unit of angular error.
Ki	Integral Gain	A tuning constant that scales the integral control action. Higher values accelerate the elimination of steady-state error but may cause oscillatory behaviour if excessive.
Kd	Derivative Gain	A tuning constant that scales the derivative control action. Higher values provide stronger damping of transient oscillations but increase sensitivity to sensor noise.
∫e(t)dt	Integral of Error	The cumulative sum of all past error values over time, representing the historical deviation of the system from the setpoint.
de(t)/dt	Derivative of Error	The instantaneous rate of change of the error signal, representing the speed at which the error is currently growing or diminishing.

Table 5.1: Definition of PID Controller Variables and Parameters

5.2.2 Discrete-Time PID Equation

Since the Arduino Mega 2560 microcontroller operates in discrete time, processing sensor data and updating motor commands at fixed intervals of $\Delta t = 10$ ms, the continuous-time PID equation must be approximated using numerical methods suitable for digital implementation. The standard approach employs Euler's method for numerical integration and the backward difference method for numerical differentiation, yielding the following discrete-time PID formulation:

$$P[n] = Kp \cdot e[n] \qquad (5.2)$$

$$I[n] = I[n-1] + Ki \cdot e[n] \cdot \Delta t \quad \textit{(Euler Integration)} \qquad (5.3)$$

$$D[n] = Kd \cdot (e[n] - e[n-1]) / \Delta t \quad \textit{(Backward Difference)} \qquad (5.4)$$

$$u[n] = P[n] + I[n] + D[n] \qquad (5.5)$$

where n denotes the current discrete time index, n−1 denotes the previous sample, and Δt is the fixed sampling interval in seconds. In the specific implementation adopted for this project, the derivative term is computed on the measurement signal rather than the error signal, a refinement discussed in detail in Section 5.7. An additional first-order Infinite Impulse Response (IIR) low-pass filter is applied to the raw derivative to attenuate high-frequency noise, yielding:

$$D\_filtered[n] = \alpha \cdot D\_raw[n] + (1 - \alpha) \cdot D\_filtered[n-1] \qquad (5.6)$$

where α is the filter coefficient (DERIV_ALPHA = 0.15 in this implementation), with smaller values providing greater noise attenuation at the cost of reduced derivative responsiveness.

5.3 Proportional Control (P)

5.3.1 Theory and Working Principle

The proportional control term constitutes the primary and most fundamental component of the PID controller. It generates a control output that is directly and instantaneously proportional to the current error signal. The underlying operating principle is straightforward: when the deviation between the desired state and the actual state is large, the controller applies a strong corrective force; as the error diminishes, the corrective force decreases proportionally. This relationship ensures that the magnitude of the control action is always commensurate with the severity of the current deviation, preventing the controller from applying excessive force when the system is already near its setpoint.

5.3.2 Mathematical Equation

$$P(t) = Kp \cdot e(t) = Kp \cdot [\theta setpoint - \theta measured(t)] \qquad (5.7)$$

In Equation 5.7, θsetpoint represents the target balance angle determined during the startup calibration procedure, and θmeasured(t) represents the current filtered tilt angle provided by the Complementary Filter. The proportional output is expressed in steps per second and is applied

directly to both stepper motors as a speed command. In the present implementation, the proportional gain Kp is set to a value of 220.0, which has been determined through empirical tuning to provide adequate restoring force without inducing high-frequency oscillations.

5.3.3 Effect on Self-Balancing Robot Stability

In the context of the self-balancing robot, the proportional term serves as the primary mechanism by which the system detects and responds to angular displacement from the vertical setpoint. When the robot begins to tilt forward by an angle of, for example, 5 degrees, the proportional controller immediately generates a motor speed command of Kp × 5 = 1100 steps per second in the forward direction, driving the wheels toward the new position of the centre of gravity and generating a counter-torque that opposes the gravitational torque responsible for the tilt. As the corrective motion reduces the tilt angle, the proportional output decreases correspondingly, preventing overcorrection.

The effectiveness of the proportional term is directly related to the value of Kp. A value that is too low results in a weak corrective response that cannot counteract the gravitational torque quickly enough, causing the robot to fall. A value that is too high causes the system to overcorrect aggressively, overshooting the setpoint and oscillating with increasing amplitude, eventually leading to instability. The relationship between Kp and system stability is therefore non-monotonic, and the selection of an appropriate value requires careful empirical tuning as described in Section 5.11.

5.3.4 Advantages and Limitations

The primary advantage of proportional control lies in its immediacy: the corrective response begins instantaneously upon the appearance of an error and is scaled linearly with the error magnitude, providing an intuitive and predictable control action. However, proportional control in isolation suffers from a fundamental structural limitation known as steady-state error or offset. Because the proportional term generates zero output when the error is zero, the system cannot maintain zero steady-state error in the presence of constant disturbances such as a shifted centre of gravity due to uneven component placement or a non-horizontal floor surface. In these situations, a small but non-zero steady-state error is required to generate sufficient proportional output to counteract the

constant disturbance, meaning that the system balances at a slightly displaced angle rather than at the true setpoint. This limitation is addressed by the integral term described in Section 5.6.

5.4 Integral Control (I)

5.4.1 Theory and Working Principle

The integral control term addresses the fundamental limitation of proportional-only control by accumulating the error signal over time and generating a control output proportional to the total historical deviation of the system from the setpoint. Unlike the proportional term, which responds exclusively to the current instantaneous error, the integral term retains a memory of all past errors and continues to increase its output as long as any non-zero error persists. This property enables the integral controller to eliminate steady-state error entirely by generating sufficient control output to overcome constant disturbances, even when the instantaneous error is very small.

The operational mechanism of the integral term can be understood intuitively as follows: if the robot consistently leans slightly forward despite the proportional and derivative corrections, the integral accumulator gradually builds up a positive value over successive control cycles. This accumulated value is scaled by Ki and added to the total PID output, progressively augmenting the forward motor speed until the residual lean is eliminated and the robot achieves a true zero-error equilibrium. The integral action effectively compensates for any systematic bias in the system, whether arising from sensor offsets, asymmetric mass distribution, or external forces.

5.4.2 Mathematical Equation

$$I(t) = Ki \cdot \int e(t)\, dt \qquad (5.8)$$

In discrete-time implementation using Euler's integration method:

$$I[n] = I[n-1] + Ki \cdot e[n] \cdot \Delta t \qquad (5.9)$$

The discrete integral accumulates the product of the error value and the sampling interval Δt at each control cycle, providing a running sum of the area under the error-time curve. In the implementation adopted for this project, the integral accumulator is clamped to the range [−INTEGRAL_CLAMP, +INTEGRAL_CLAMP], where INTEGRAL_CLAMP is set to 40

percent of MAX_SPEED (800 steps per second), to prevent the phenomenon of integral windup discussed in Section 5.13.

5.4.3 Steady-State Error Correction

The elimination of steady-state error is the defining functional contribution of the integral term. Consider a scenario in which the battery pack of the self-balancing robot is positioned slightly toward the rear, shifting the centre of gravity behind the wheel axis. Under proportional-only control, the robot would maintain balance with a small but persistent forward lean, just sufficient to generate the proportional output necessary to counteract the gravitational torque from the rear-biased CoG. With the integral term active, this persistent forward lean continuously increments the integral accumulator. After a finite number of control cycles, the accumulated integral output reaches a sufficient magnitude to provide the additional corrective force that eliminates the forward lean, driving the actual balance angle back to the calibrated setpoint.

5.4.4 Advantages and Limitations

The integral term provides the essential capability of zero steady-state error in the presence of constant or slowly varying disturbances. However, it introduces several significant challenges. The most serious is integral windup, which occurs when the control output saturates at its maximum value (for example, during a large perturbation or fall event) while the integrator continues to accumulate error unchecked. When the saturation condition is eventually resolved, the wound-up integral generates an excessive control output in the opposite direction, causing severe overshoot and potentially inducing oscillatory instability. The anti-windup mechanism implemented in this project addresses this limitation through conditional integration suspension, as described in Section 5.13. Additionally, an excessively large value of Ki increases the integral's contribution too rapidly, causing oscillation or instability. The integral gain must therefore be tuned conservatively, and the integral is typically the last of the three PID gains to be adjusted during the tuning process.

5.5 Derivative Control (D)

5.5.1 Theory and Working Principle

The derivative control term provides a predictive, anticipatory action that responds to the rate of change of the error signal rather than its instantaneous magnitude. By computing the derivative of the error with respect to time, the derivative term effectively forecasts the future trend of the error based on its current trajectory: if the error is diminishing rapidly, the derivative term generates a braking force that reduces the aggressiveness of the proportional response, preventing overshoot; if the error is growing rapidly, the derivative term augments the corrective action to counteract the accelerating deviation. The derivative term therefore functions analogously to a mechanical damper or shock absorber in a spring-mass system, opposing rapid changes in the system state and dissipating oscillatory energy.

In the context of the self-balancing robot, the derivative term plays a particularly critical role in suppressing the oscillatory behaviour that would otherwise result from the proportional term acting on the inherently resonant inverted pendulum dynamics. Without derivative damping, the robot would oscillate with growing amplitude around the balance point, eventually falling. The derivative term prevents this by generating a force that opposes the angular velocity of the robot, effectively adding damping to the underdamped oscillatory mode of the closed-loop system.

5.5.2 Mathematical Equation and Derivative-on-Measurement Formulation

The standard derivative term in continuous time is expressed as:

$$D(t) = Kd \cdot de(t)/dt \qquad (5.10)$$

In discrete-time implementation using the backward difference approximation:

$$D[n] = Kd \cdot (e[n] - e[n-1]) / \Delta t \qquad (5.11)$$

However, a significant practical refinement is employed in this implementation. Rather than computing the derivative of the error signal, the derivative is computed on the measurement signal directly, a technique known as Derivative-on-Measurement (DoM). This modification is expressed as:

$$D[n] = -Kd \cdot (\theta measured[n] - \theta measured[n-1]) / \Delta t \qquad (5.12)$$

Since the setpoint (balance angle) is a constant value during steady-state operation, the derivative of the error is equal to the negative derivative of the measurement, making Equations 5.11 and 5.12 mathematically equivalent in the absence of setpoint changes. However, the critical advantage of the DoM formulation manifests when the setpoint is suddenly changed, as occurs during locomotion commands (forward, reverse, turning). In the standard formulation, a step change in the setpoint produces a large impulsive spike in de(t)/dt, known as derivative kick, which causes a violent motor response that can destabilise the robot. The DoM formulation is immune to this phenomenon because the measurement changes continuously and smoothly, regardless of setpoint discontinuities.

5.5.3 Derivative Low-Pass Filter

The differentiation operation inherently amplifies high-frequency noise components present in the tilt angle signal. Since the angle estimate is derived from a noisy IMU sensor, the raw derivative signal contains significant noise at frequencies far above the bandwidth of the robot's dynamics. Without filtering, this noise is amplified by the derivative gain Kd and applied to the motors as a rapidly fluctuating speed command, causing audible chattering, increased motor heating, and degraded balance quality. To mitigate this effect, a first-order IIR low-pass filter is applied to the raw derivative term:

$$D\_filtered[n] = \alpha \cdot D\_raw[n] + (1 - \alpha) \cdot D\_filtered[n-1] \qquad (5.13)$$

where α = DERIV_ALPHA = 0.15, resulting in a low-pass filter that heavily attenuates the high-frequency noise while preserving the low-frequency derivative signal that carries useful information about the robot's angular velocity.

5.5.4 Advantages and Limitations

The derivative term provides essential damping that suppresses oscillatory behaviour and improves transient response quality. By opposing the rate of change of the error, it reduces overshoot, shortens settling time, and enables the use of a higher proportional gain than would otherwise be stable, resulting in a more responsive and robust controller. The principal limitation of the derivative term is its sensitivity to measurement noise, as discussed above. This sensitivity necessitates the use of the low-pass filter described in Section 5.7.3, which introduces a trade-off between noise rejection and derivative responsiveness. Furthermore, the derivative term does not

contribute to steady-state error correction and is ineffective in the presence of constant disturbances. Its role is exclusively concerned with the transient dynamics of the system.

5.6 Combined PID Controller

5.6.1 Interaction Between P, I, and D Terms

The complete PID control output is the algebraic sum of the three individual terms, each contributing its distinct corrective action simultaneously:

$$u[n] = Kp \cdot e[n] + Ki \cdot I[n] + Kd \cdot D\_filtered[n] \qquad (5.14)$$

The three terms interact in a complementary manner that collectively addresses all aspects of the system's dynamic response. The proportional term provides the primary restoring force, reacting immediately and proportionally to the current error. The integral term accumulates historical error information and provides additional corrective force to eliminate residual steady-state deviations. The derivative term provides predictive damping, opposing the rate of change of the error to prevent overshoot and suppress oscillations. Together, the three terms enable the controller to achieve a balance between responsiveness, accuracy, and stability that no single term can provide individually.

The interaction between the terms also manifests in their mutual influence on each other's effective contribution. When the proportional gain is increased to improve response speed, the resulting increase in oscillation tendency makes the derivative term more important for stability. When the integral gain is increased to eliminate steady-state error more aggressively, the increased risk of integral windup during transients requires careful implementation of anti-windup logic. The tuning of the three gains is therefore an interdependent process, and adjustment of any single gain may require compensatory adjustment of the others.

5.7 PID Control Block Diagram

The closed-loop control architecture of the self-balancing robot is depicted in Figure 5.2. The block diagram illustrates the complete signal flow from the reference setpoint through the PID controller, the stepper motor actuators, the robot plant, the MPU6050 sensor, and the Complementary Filter feedback path back to the summation junction at the controller input.

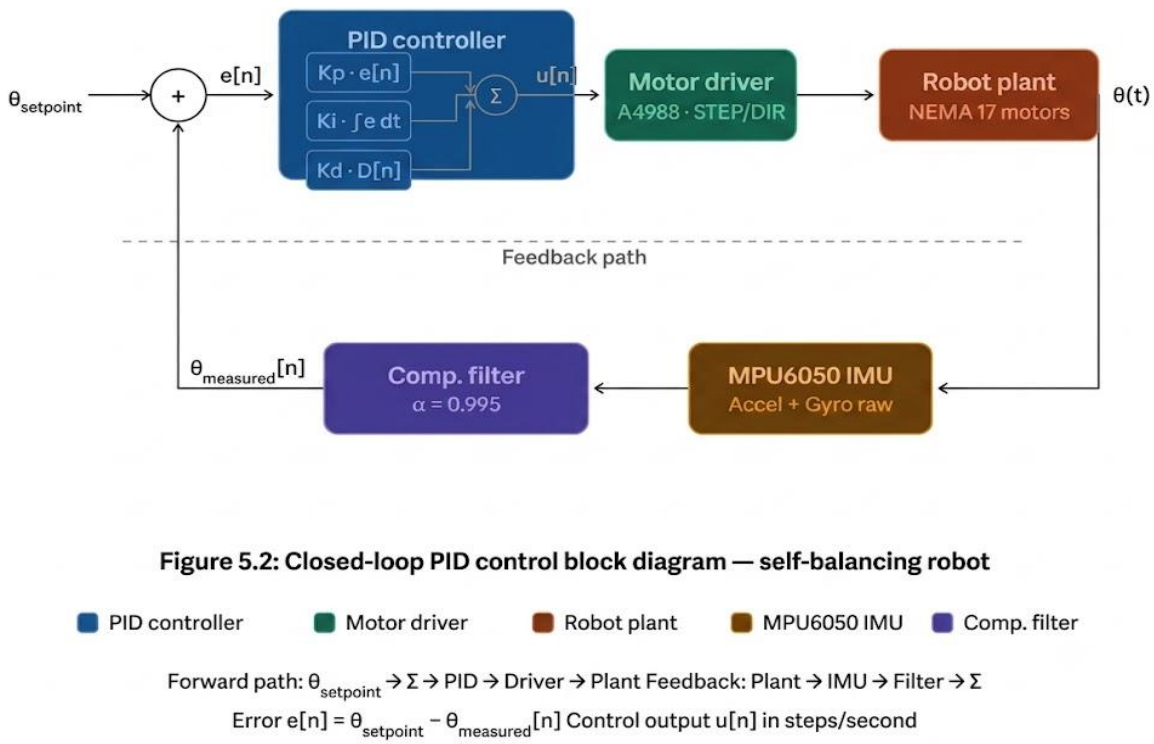


Figure 5.2: Closed-loop PID control block diagram — self-balancing robot

Figure 5.2: Closed-Loop PID Control Block Diagram for the Self-Balancing Robot

In the block diagram, the reference input θsetpoint is the calibrated balance angle established during the startup calibration procedure. The error signal e[n] is formed at the summing junction as the difference between the setpoint and the feedback signal θmeasured[n] returned by the Complementary Filter. The PID controller block processes the error signal through its three parallel computation paths (P, I, and D branches) and sums their outputs to produce the motor speed command u[n]. The motor driver block translates this speed command into STEP and DIR pulses that drive the NEMA 17 stepper motors. The mechanical response of the robot (its tilt angle) is measured by the MPU6050 IMU, whose raw sensor data is processed by the Complementary Filter to produce the feedback signal θmeasured[n], closing the loop

5.8 PID Tuning

5.8.1 Manual Tuning

Manual tuning is the most fundamental approach to PID gain selection and involves the systematic adjustment of each gain through direct observation of the system's response. The process typically begins with all three gains set to zero, after which each gain is individually increased from zero

while the system is operated and its behaviour is observed. The tuning engineer must possess a thorough understanding of the effect of each gain on the system's response, as described in Section 5.10, in order to interpret the observed behaviour and determine the appropriate direction and magnitude of gain adjustment.

5.8.2 Trial-and-Error Tuning

Trial-and-error tuning is an iterative refinement of manual tuning in which each gain is adjusted incrementally, the system is tested under representative operating conditions, the response is evaluated against performance criteria, and the gain is further adjusted based on the evaluation. This process continues until a satisfactory combination of responsiveness, stability, and accuracy is achieved. Trial-and-error tuning is the most commonly employed approach for embedded control applications where a precise mathematical model of the plant is unavailable, as is the case for the self-balancing robot developed in this project.

5.8.3 Ziegler–Nichols Tuning Method

The Ziegler–Nichols method is a systematic, model-free tuning procedure developed by John G. Ziegler and Nathaniel B. Nichols in 1942 and remains one of the most widely referenced classical tuning methods. The procedure is applicable when a mathematical model of the plant is unavailable and involves the determination of two experimentally measured parameters: the ultimate gain Ku and the ultimate period Pu.

The procedure begins by setting Ki = 0 and Kd = 0 and increasing Kp from zero until the closed-loop system exhibits sustained, constant-amplitude oscillations without diverging or decaying. The value of Kp at which this condition is observed is defined as the ultimate gain Ku, and the period of the sustained oscillations is defined as the ultimate period Pu. The Ziegler–Nichols tuning rules then prescribe the following initial gain values:

$$Kp = 0.6 \cdot Ku \qquad (5.15)$$

$$Ki = 2 \cdot Kp / Pu \qquad (5.16)$$

$$Kd = Kp \cdot Pu / 8 \qquad (5.17)$$

These values are intended as starting points rather than optimal final values and typically require further empirical refinement. In the context of the self-balancing robot, the application of the Ziegler–Nichols method is complicated by the inherent instability of the inverted pendulum, which

makes the sustained-oscillation condition difficult to achieve safely without risking mechanical damage. The method was therefore used as a preliminary guideline, with subsequent empirical refinement to the values documented in Section 5.12.

5.8.4 Practical Tuning Procedure for the Self-Balancing Robot

The following sequential tuning procedure was applied in this project to determine the final PID gain values:

- Step 1 – Proportional Gain (Kp): With Ki = 0 and Kd = 0, Kp was increased incrementally from zero. At low values, the robot leaned excessively in response to any perturbation and fell. As Kp was increased, the corrective response became stronger, and the robot began to resist perturbations. The optimal range for Kp was identified as the interval in which the robot responded vigorously to perturbations without exhibiting high-frequency oscillation. The final value of Kp = 220.0 was determined to provide adequate restoring force with acceptable transient behaviour prior to derivative tuning.
- Step 2 – Derivative Gain (Kd): With Kp fixed at 220.0 and Ki = 0, Kd was increased from zero. At Kd = 0, the robot exhibited significant oscillatory behaviour with a period of approximately 1–2 seconds, corresponding to the underdamped closed-loop mode. As Kd was increased, the amplitude of oscillations decreased progressively. The final value of Kd = 2.5 was selected as a compromise between oscillation suppression and the onset of noise amplification, which manifested as audible motor chattering at higher derivative gain values.
- Step 3 – Integral Gain (Ki): With Kp = 220.0 and Kd = 2.5, Ki was increased from zero to address the residual steady-state lean observed under proportional-derivative control. Values of Ki below 0.1 produced negligible improvement, while values above 0.5 introduced slow oscillatory modes with periods of approximately 7–10 seconds. The final value of Ki = 0.3 was determined to provide effective steady-state correction without inducing low-frequency instability.
- Step 4 – Verification and Refinement: The robot was tested under a variety of conditions including step perturbations, inclined surfaces, and payload variations to verify the robustness of the selected gains. Minor incremental adjustments were made to Kp and Kd to improve the response to large perturbations, yielding the final gain set documented in Section 5.12.

5.9 PID Implementation in the Self-Balancing Robot

5.9.1 Angle Acquisition from MPU6050

At the commencement of each 100 Hz control cycle, the firmware reads raw accelerometer and gyroscope data from the MPU6050 Inertial Measurement Unit via the I²C interface at 400 kHz. The raw data undergoes offset correction using the bias values determined during the startup calibration procedure, followed by conversion to engineering units: accelerometer readings are expressed in units of gravitational acceleration (g) using a scale factor of 16384 LSB/g, and gyroscope readings are expressed in degrees per second using a scale factor of 131 LSB/(°/s). The corrected sensor readings are then passed to the Complementary Filter function, which fuses the gyroscope angular rate and the accelerometer-derived tilt angle using the fusion formula presented in Equation 5.6 of Chapter 5 (Complementary Filter), producing a clean, drift-free angle estimate θmeasured.

5.9.2 Error Computation

The error signal for the current control cycle is computed as the difference between the calibrated balance setpoint and the filtered tilt angle:

$$e[n] = \theta setpoint - \theta measured[n] \qquad (5.18)$$

The balance setpoint θsetpoint is established during the startup calibration procedure as the mean tilt angle of the robot measured over 1000 sensor samples while held stationary in the upright position. This adaptive setpoint determination ensures that the controller targets the robot's actual physical upright angle rather than an assumed zero-degree ideal, compensating for systematic misalignment between the MPU6050 sensor mounting plane and the true vertical axis of the chassis.

5.9.3 PID Computation and Motor Speed Output

The three PID terms are computed sequentially within the pidUpdate() function. The proportional term P[n] = Kp × e[n] is computed first as it requires no state information from previous cycles beyond the current error. The integral accumulator I[n] is updated by adding Ki × e[n] × Δt to the previous accumulator value stored in the global variable integral, subject to the anti-windup clamping condition described in Section 5.13. The raw derivative D_raw[n] is computed as the negative rate of change of the measurement: D_raw[n] = −(θmeasured[n] − θmeasured[n−1]) / Δt,

and the filtered derivative D_filtered[n] is obtained through the IIR filter of Equation 5.13. The total PID output u[n] is then constrained to the range [−MAX_SPEED, +MAX_SPEED] = [−2000, +2000] steps per second.

5.9.4 Velocity Damping Augmentation

In addition to the standard PID output, the firmware applies an additional velocity damping correction that subtracts a term proportional to the instantaneous angular rate from the final motor speed command:

$$u\_final[n] = u[n] - velocityDamping \cdot (\theta measured[n] - \theta measured[n-1]) / \Delta t \quad (5.19)$$

where velocityDamping = 0.15. This additional damping term provides supplementary suppression of the slow 7–10 second oscillatory mode that persists under PID control alone, without the noise sensitivity associated with increasing Kd directly. The final damping-corrected output u_final[n] is applied to both stepper motors via the setMotorSpeed() function.

5.9.5 Final Tuned PID Parameter Values

Parameter	Symbol	Value	Physical Interpretation
Proportional Gain	**Kp**	220.0	Provides primary restoring torque proportional to tilt angle.
Integral Gain	**Ki**	0.3	Eliminates steady-state lean from mass asymmetry.
Derivative Gain	**Kd**	2.5	Damps oscillations;

			opposes angular velocity.
Integral Clamp	**INTEGRAL_CLAMP**	800 steps /s	Prevents integral windup during fall events.
Derivative Filter	**DERIV_ALPHA**	0.15	Attenuates IMU noise in derivative calculation.
Velocity Damping	**velocityDamping**	0.15	Additional damping of slow oscillatory modes.
Complementary Filter	**COMP_FILTER_ALPHA**	0.995	Sensor fusion weight; 99.5% gyroscope, 0.5% accelerometer.

Table 5.3: Final Tuned PID Implementation Parameters for the Self-Balancing Robot

5.10 Advantages and Limitations of PID Control

The application of PID control to the self-balancing robot presents a number of significant operational advantages alongside inherent architectural limitations. These are summarised in Table 5.4 below.

Advantages	Limitations
Conceptually simple and well-understood, enabling straightforward implementation on resource-constrained 8-bit microcontrollers such as the ATmega2560.	Assumes linear plant dynamics; performance degrades at large angular displacements where the inverted pendulum exhibits significant nonlinearity.

Computationally efficient: the complete PID computation executes in microseconds, leaving ample CPU headroom for sensor communication and motor pulse generation.	Three interdependent gain parameters require careful empirical tuning; no systematic closed-form solution exists for nonlinear plants.
Robust to moderate variations in system parameters and external disturbances, making it well-suited to the variable-load operating conditions of the robot.	The integral term introduces potential for windup instability during actuator saturation, requiring additional anti-windup implementation complexity.
Eliminates steady-state error in the presence of constant disturbances through integral action, ensuring the robot maintains its upright position without persistent angular offset.	The derivative term amplifies sensor noise, necessitating additional filtering that reduces derivative responsiveness and introduces phase lag.
No mathematical model of the plant is required for implementation or tuning; the controller treats the plant as a black box and is tuned through empirical observation.	Single-axis control: the standard PID controller addresses only the pitch axis. Yaw control requires additional mechanisms (differential setpoint in this implementation).
Extensive industrial and academic precedent provides a well-established foundation of design guidelines, tuning heuristics, and performance prediction methods.	Cannot inherently optimise multiple performance objectives simultaneously; a trade-off between response speed, damping, and steady-state accuracy is always present.

Table 5.4: Advantages and Limitations of PID Control in the Self-Balancing Robot Application

CHAPTER 6 :- Software Architecture & Code Flow

This chapter provides a detailed technical description of the software architecture and execution flow of the self-balancing robot firmware. The firmware is structured as a single-file, interrupt-free embedded C++ program targeting the Arduino Mega 2560 (ATmega2560) platform. All real-time constraints are managed through explicit software timing gates, and all hardware interactions are performed through precisely sequenced function calls within a deterministic main execution loop. The following sections describe the architecture of the main control loop, provide detailed specifications of each functional module, and document the complete set of tunable system parameters.

6.1 Main Loop Architecture

The firmware's execution architecture is organised into two distinct temporal tiers within the Arduino loop() function. This two-tier structure ensures that the most time-critical operation — stepper motor pulse generation — receives the highest possible update frequency, while the computationally heavier sensor processing and control calculation are performed at a precisely controlled 100 Hz rate.

6.1.1 Tier 1: High-Frequency Motor Pulse Execution

On every iteration of the loop() function, regardless of the control loop timing gate, the following calls are executed unconditionally:

```
motor1.runSpeed();
motor2.runSpeed();
```

These calls instruct the AccelStepper library to evaluate whether a step pulse is due for each motor and, if so, to toggle the corresponding STEP pin on the Arduino. Since the Arduino Mega 2560 operates at 16 MHz and the loop() function executes in the order of tens of microseconds, the effective step pulse resolution far exceeds the minimum achievable by a fixed 100 Hz gate. This high-frequency polling ensures smooth, jitter-free motor rotation at all commanded speeds.

6.1.2 Tier 2: 100 Hz Control Loop (10 ms Gate)

The control computation is gated by a microsecond-precision timer check using the micros() function. When the elapsed time since the previous control cycle exceeds **LOOP_PERIOD_US** (10,000 microseconds), the following sequence of operations is executed:

Step	Operation	Description
1	Elapsed Time Computation	The actual elapsed time dt is computed in seconds from the micros() counter. This measured dt is used in all filter and PID calculations rather than the assumed nominal value of 0.01 s, ensuring that any timing jitter does not accumulate as angle estimation error.
2	dt Clamping	The computed dt value is clamped to the range [0.001 s, 0.05 s]. The lower bound prevents division-by-zero errors. The upper bound (equivalent to 20 Hz) prevents large derivative spikes that would occur if the control loop stalled due to an I2C timeout or other transient anomaly.
3	Sensor Data Acquisition	Raw accelerometer and gyroscope data are read from the MPU6050 via a burst I2C read. Calibration offsets are subtracted and the values are converted to engineering units (g and °/s).
4	Complementary Filter	The corrected sensor readings are passed to the complementaryFilter() function, which fuses the gyroscope angular rate and the accelerometer tilt angle to produce a clean, drift-free angle estimate.
5	Angular Deviation	The angular deviation from the balance setpoint is computed as: angleDev = filteredAngle − balanceSetpoint.
6	Fall Detection	The fall detection state machine evaluates |angleDev| against FALL_ANGLE and RECOVERY_ANGLE thresholds, enabling or disabling the motor drivers and resetting PID state as required.
7	PID Computation	If the robot is in the BALANCING state, the pidUpdate() function is called with the balance setpoint, the current filtered angle, and dt. The function returns a motor speed command.
8	Velocity Damping	An additional velocity damping correction term is subtracted from the PID output: pidOutput −= velocityDamping × angleRate. This provides supplementary damping of low-frequency oscillations beyond the PID derivative term.

9	Motor Speed Command	The final corrected PID output is applied to both motor drivers via setMotorSpeed(), followed by an immediate runSpeed() call to ensure the new speed command takes effect within the current cycle.

Table 6.1: 100 Hz Control Loop — Sequential Execution Steps

The overall software execution flow from power-on through continuous operation is illustrated conceptually in the following sequence:

```
POWER ON → setup()
   ├── initMPU()          : Configure MPU6050 registers
   ├── calibrate()        : Compute offsets and balance setpoint
   └── enableMotors()     : Assert ENABLE LOW on A4988 drivers


loop()  [repeats indefinitely]
   ├── motor1.runSpeed()          [every iteration — no gate]
   ├── motor2.runSpeed()          [every iteration — no gate]
   └── if (elapsed ≥ 10 ms)      [100 Hz gate]
       ├── compute dt
       ├── readSensor()
       ├── complementaryFilter()
       ├── compute angleDev
       ├── fallDetection()
       ├── if (!isFallen)
       │   ├── pidUpdate()
       │   ├── apply velocityDamping
       │   └── setMotorSpeed()
       └── motor1.runSpeed() / motor2.runSpeed()
```

Code Listing 6.1: Firmware Execution Flow (Pseudocode)

6.2 Function Descriptions

This section provides detailed specifications for each functional module implemented within the firmware. Each function description includes its purpose, input parameters, internal logic, and output.

6.2.1 initMPU()

Purpose: Initialises the MPU6050 sensor by writing configuration values to its internal registers over the I2C bus. This function is called once during setup() and must complete before any sensor data is read.

Register	Value	Effect
PWR_MGMT_1 (0x6B)	0x00	Clears the SLEEP bit to wake the sensor from its power-on sleep state. Selects the internal 8 MHz oscillator as the clock source.
SMPLRT_DIV (0x19)	0x09	Sets the sample rate divider to 9, producing a sensor output rate of 1000 Hz ÷ (1 + 9) = 100 Hz, matching the control loop frequency exactly.
CONFIG (0x1A)	0x03	Enables the on-chip Digital Low-Pass Filter (DLPF) at level 3, setting the gyroscope bandwidth to 44 Hz. Motor vibrations above 50 Hz are attenuated before reaching the firmware.
GYRO_CONFIG (0x1B)	0x00	Selects ±250°/s full-scale range for the gyroscope (FS_SEL = 0), providing a sensitivity of 131 LSB per °/s — the highest available resolution.
ACCEL_CONFIG (0x1C)	0x00	Selects ±2g full-scale range for the accelerometer (AFS_SEL = 0), providing a sensitivity of 16384 LSB per g — optimised for gravity-vector measurement.

Table 6.2: MPU6050 Register Configuration — initMPU()

6.2.2 calibrate()

Purpose: Performs the automated startup calibration procedure described in Section 5.5. Computes gyroscope and accelerometer offset values, determines the balance setpoint angle from the robot's physical upright position, and initialises all PID state variables to a zero-error starting condition.

Inputs: None (reads directly from the MPU6050 via I2C).

Outputs: Writes to global variables gyroX_offset, accelY_offset, accelZ_offset, and balanceSetpoint. Initialises filteredAngle, prevMeasurement, and lastAngle to balanceSetpoint. Sets integral and prevDerivative to zero.

Key implementation detail: The Z-axis accelerometer offset is computed as the sample mean minus 16384 LSB (representing 1g), so that the corrected Z reading equals zero when the sensor is perfectly aligned with the gravitational vector. This ensures that the atan2() computation in the Complementary Filter returns zero degrees at the true vertical position.

6.2.3 readSensor()

Purpose: Reads raw sensor data from the MPU6050 and returns a SensorData structure containing the calibrated accelerometer Y and Z values in units of g, and the calibrated gyroscope X value in units of °/s.

Inputs: None. Uses global calibration offset variables.

Outputs: Returns a SensorData struct with fields accelY (float, units: g), accelZ (float, units: g), and gyroX (float, units: °/s).

Implementation: A single I2C burst read of 14 bytes (7 × 16-bit words) starting at register MPU_ACCEL_XOUT_H (0x3B) is performed using the mpuReadWords() helper. This atomic read ensures all seven sensor values are sampled at the same instant. Raw values are converted to physical units by subtracting calibration offsets and dividing by the respective scale factors.

6.2.4 complementaryFilter()

Purpose: Implements the Complementary Filter sensor fusion algorithm to compute a stable, drift-free tilt angle estimate from the fused gyroscope and accelerometer measurements.

Parameter	Type	Description
gyroRate	float	Calibrated gyroscope X-axis angular rate in degrees per second.
accelY	float	Calibrated accelerometer Y-axis measurement in units of g.
accelZ	float	Calibrated accelerometer Z-axis measurement in units of g.
dt	float	Elapsed time since the previous control cycle in seconds.
Return value	float	The updated filtered tilt angle in degrees, stored in the global variable filteredAngle.

Table 6.3: complementaryFilter() — Parameter Specification

The function first computes the gyroscope-predicted angle by integrating the angular rate over dt. It then computes the accelerometer-derived static angle using the atan2f() function applied to the Y and Z accelerometer components. The final filtered angle is computed as the weighted blend of these two estimates using the coefficient COMP_FILTER_ALPHA = 0.995.

6.2.5 pidUpdate()

Purpose: Computes the PID control output for the current control cycle and returns a motor speed command value in steps per second, clamped to the range [−MAX_SPEED, +MAX_SPEED].

Parameter	Type	Description
setpoint	float	The target balance angle in degrees, equal to balanceSetpoint.
measurement	float	The current filtered tilt angle in degrees, as returned by complementaryFilter().

dt	float	Elapsed time since the previous control cycle in seconds, used for numerical integration and differentiation.
Return value	float	The PID output in steps per second, constrained to [−MAX_SPEED, +MAX_SPEED].

Table 6.4: pidUpdate() — Parameter Specification

The function internally maintains three persistent state variables across successive calls: integral (the accumulated integral term), prevMeasurement (the measurement value from the previous cycle, used for derivative computation), and prevDerivative (the filtered derivative value from the previous cycle, used by the IIR low-pass filter).

The anti-windup logic evaluates whether the current output is saturated at MAX_SPEED or −MAX_SPEED and whether the error signal is driving the output further into saturation. If both conditions are true, the integral accumulation is suspended for the current cycle to prevent windup. The integral remains hard-clamped to ±INTEGRAL_CLAMP at all times regardless of the anti-windup condition.

6.2.6 setMotorSpeed()

Purpose: Applies the computed motor speed command to both stepper motor driver objects, incorporating any necessary direction inversion for the physical motor mounting configuration.

Inputs: speed (float) — the desired motor speed in steps per second, in the range [−MAX_SPEED, +MAX_SPEED].

Implementation: The input speed is first clamped to the valid range using constrain(). If the compile-time constant INVERT_MOTOR1 is set to true, the speed applied to motor1 is negated to compensate for the physical reversal of that motor's mounting direction on the chassis. This ensures that a positive speed command drives both wheels in the same physical direction. Both motors receive the same speed magnitude, as differential steering is handled separately by the setpoint offset mechanism described in Section 5.7.

6.2.7 Fall Detection State Machine

Purpose: Monitors the angular deviation of the robot from its balance setpoint and manages the isFallen boolean state flag and the PIN_ENABLE hardware output to protect the motor drivers during fall events and re-enable them upon recovery.

The state machine is implemented as a conditional branch within the 100 Hz control loop gate. Two independent threshold comparisons govern the state transitions:

Condition Evaluated	State Transition	Hardware and Software Actions
(!isFallen) AND (|angleDev| > FALL_ANGLE)	BALANCING → FALLEN	isFallen = true; PIN_ENABLE → HIGH (drivers off); integral, prevMeasurement, prevDerivative reset to zero.
(isFallen) AND (|angleDev| < RECOVERY_ANGLE)	FALLEN → BALANCING	isFallen = false; PIN_ENABLE → LOW (drivers on); filteredAngle, prevMeasurement, integral, prevDerivative re-initialised from current angle.

Table 6.5: Fall Detection State Machine — Transition Logic

The complete PID state reset performed during the BALANCING → FALLEN transition is essential to prevent erratic motor behaviour upon recovery. Without this reset, the integral accumulator would retain a large wound-up value from the fall event, causing the motors to command a large corrective speed immediately upon re-enabling — potentially causing a second fall in the opposite direction. The 25-degree hysteresis between the two thresholds (FALL_ANGLE = 35° and RECOVERY_ANGLE = 10°) prevents rapid oscillation between states during a fall in progress.

6.3 Tunable System Parameters

The firmware exposes a set of compile-time tunable constants that govern all aspects of the system's behaviour. These parameters are defined in the header section of the source file as preprocessor macros and global floating-point variables. The following table provides a complete reference for all tunable parameters, their default values, units, and tuning guidance.

Parameter	Default Value	Description and Tuning Guidance
Kp	220.0	Proportional gain. Increase to improve response speed and reduce lean; decrease if rapid oscillations develop around the setpoint.
Ki	0.3	Integral gain. Increase to correct persistent steady-state lean or floor drift; decrease if oscillation or overshoot increases after correction events.
Kd	2.5	Derivative gain. Increase gradually to damp long-period oscillations (7–10 s rocking); excessive values amplify sensor noise and cause audible motor chattering.
COMP_FILTER_ALPHA	0.995	Complementary filter blending coefficient. Decrease toward 0.98 if angle drifts over minutes; increase toward 0.999 if vibration noise causes spurious corrections.
DERIV_ALPHA	0.15	IIR low-pass filter coefficient for the derivative term. Increase toward 1.0 to reduce noise filtering (more responsive D); decrease to increase noise attenuation.

FALL_ANGLE	35.0 degrees	Angular deviation beyond which the robot is declared fallen and motors are disabled. Reduce for faster hardware protection; increase if the robot must survive large external disturbances.
RECOVERY_ANGLE	10.0 degrees	Angular deviation below which recovery is permitted after a fall. Must be significantly less than FALL_ANGLE to maintain hysteresis and prevent re-enable oscillation.
MAX_SPEED	2000 steps/s	Maximum motor speed in steps per second. Reduce if motors stall under rapid direction reversals. Increase for heavier robots requiring greater peak torque.
MAX_ACCEL	4000 steps/s²	Maximum stepper motor acceleration. Used only for AccelStepper initialisation; the balancing control uses constant-speed mode rather than position-profile mode.
INTEGRAL_CLAMP	800 (40%) steps/s	Absolute maximum value of the integral accumulator. Expressed as 40% of MAX_SPEED. Reduce if severe overshoot occurs after recovery; increase if steady-state correction is insufficient.
BALANCE_ANGLE_OFFSET	0.0 degrees	Static offset added to the calibrated balance setpoint angle. Use to compensate for systematic center-of-mass offsets from asymmetric component placement or payload.
CALIB_SAMPLES	1000 samples	Number of sensor samples collected during startup calibration. Increasing this value improves offset

		accuracy but extends the calibration duration.
velocityDamping	0.15	Additional velocity damping coefficient applied post-PID. Increase to suppress slow rocking oscillations without altering Kd; excessive values cause sluggish disturbance recovery.
LOOP_PERIOD_US	10000 µs	Control loop period in microseconds. Corresponds to a 100 Hz update rate. Modifying this value requires corresponding adjustment of COMP_FILTER_ALPHA and all time-dependent PID gains.

Table 6.6: Complete Tunable Parameter Reference

6.4 Summary

This chapter has presented a comprehensive description of the software architecture and code flow of the self-balancing robot firmware. The two-tier execution architecture — comprising a high-frequency motor pulse tier and a gated 100 Hz control tier — ensures optimal performance of both the stepper motor actuation and the real-time control computation on the ATmega2560 platform. Detailed functional specifications were provided for each firmware module, covering sensor initialisation, calibration, data acquisition, sensor fusion, PID computation, motor actuation, and hardware safety. The complete tunable parameter reference documented in Table 6.6 provides a systematic basis for system commissioning, performance optimisation, and adaptation to alternative hardware configurations. Together, Chapters 5 and 6 constitute a complete technical description of the embedded software system implemented in this project.

CHAPTER 7 :-FUTURE IMPROVEMENTS

The current prototype serves as a robust foundation for an autonomous self-balancing vehicle. To transition this from a laboratory prototype to a sophisticated, intelligent mobile platform, several technical upgrades and feature expansions are recommended to enhance performance and autonomy.

7.1 Wireless Teleoperation

The current serial-based command interface relies on a physical USB connection, which inherently restricts the robot's range of motion. Integrating an HC-05/HC-06 Bluetooth module or an ESP8266/ESP32 Wi-Fi module would enable remote control via mobile applications or custom web dashboards. This upgrade would facilitate real-time telemetry streaming, allowing for the monitoring of tilt angle, PID output, and battery health without the mechanical interference caused by tethered cables, which is critical for successful field testing.

7.2 Autonomous Obstacle Avoidance

By utilizing the abundant GPIO pins and processing headroom of the Arduino Mega 2560, the robot can be equipped with proximity sensors such as HC-SR04 ultrasonic or infrared sensors. In this implementation, the control loop would periodically poll these sensors to detect environmental obstacles. Upon detection, the system logic would be configured to dynamically adjust the PID setpoint, allowing the robot to autonomously halt movement or initiate a corrective reverse maneuver to maintain its equilibrium while avoiding collisions.

7.3 Advanced Control Algorithms

While the existing PID control is highly effective for basic balancing, integrating a Linear Quadratic Regulator (LQR) approach could provide superior performance by optimizing control inputs based on an integrated cost function. An LQR implementation would allow the system to manage complex multi-variable states—such as balancing while accelerating or carrying varying payloads—more efficiently than a standard PID controller, ultimately providing smoother and more aggressive maneuvering capabilities.

7.4 Adaptive and Auto-Tuning PID

Currently, the PID gains (Kp, Ki, and Kd) are hard-coded and require manual tuning, which limits flexibility. Implementing an auto-tuning algorithm, such as a simplified Ziegler-Nichols method or a Fuzzy Logic Controller, would allow the system to dynamically adapt to its physical environment. This capability is particularly advantageous if the robot’s physical mass distribution changes, as the controller could automatically recalibrate to maintain stability under varying conditions.

7.5 IMU Sensor Fusion Upgrades

Although the current Complementary Filter is computationally efficient, higher precision can be achieved through more advanced sensor fusion techniques. Potential upgrades include offloading sensor fusion calculations directly to the MPU6050 hardware via a Digital Motion Processor (DMP) or implementing a full Kalman Filter for superior noise rejection. Any migration to these complex filtering methods must be carefully profiled to ensure the control loop frequency remains strictly within the required 10ms (100Hz) timing constraint.

7.6 Power Efficiency and Management

Because stepper motors are power-intensive, future iterations should incorporate a more robust power management system. This includes adding a voltage divider circuit to the analog inputs for real-time battery status monitoring. Furthermore, implementing a "Safe-Park" mode would allow the system to detect low-battery states and trigger a controlled shutdown, where the motors are gradually brought to a halt rather than losing power abruptly while the robot is balancing.

7.7 Data Logging

Integrating a microSD card module via the SPI interface would enable the high-frequency logging of sensor and control data during operation. This diagnostic utility would allow for the export of data to environments such as MATLAB or Python for post-run analysis. By performing this analysis, engineers can empirically identify oscillation trends and achieve the precision tuning of PID parameters, leading to a more refined and stable system performance.

CHAPTER 8:-CONCLUSION

The completion of this project represents a successful convergence of mechanical design, electronic integration, and modern control theory. By engineering an autonomous two-wheel self-balancing robot from the ground up, we have effectively demonstrated that an inherently unstable inverted pendulum mechanism can be stabilized through precise, high-frequency closed-loop control. The transition from theoretical modeling to a functional physical prototype has provided valuable insights into the complexities of real-time embedded systems, particularly the challenges associated with sensor noise, signal drift, and the necessity of maintaining strict loop timing.

The implementation of the 100Hz PID control loop, paired with a robust complementary filter, proved to be the cornerstone of the system's stability. Throughout the development lifecycle, it was observed that the synergy between software-based filtering and the mechanical rigidity of the chassis was essential for achieving balance. While the initial tuning process presented significant challenges in terms of managing oscillations and establishing the correct gains, the final iteration of the PID algorithm provided a highly responsive and stable performance, capable of recovering from external disturbances and executing controlled linear and rotational translations.

Ultimately, this project serves as a scalable platform for advanced robotics research. Beyond achieving the primary objective of self-balancing, the architecture developed here provides a solid foundation for more complex autonomous behaviors. Whether through the integration of advanced state-space control algorithms or the addition of environmental perception sensors, the success of this prototype validates the effectiveness of the hardware and software design choices made. The knowledge gained in sensor fusion, control systems, and stepper motor synchronization contributes significantly to a broader understanding of mobile robotics, setting the stage for future developments in autonomous navigation and adaptive control systems.

REFERNCES

1. Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall. This text provided the fundamental mathematical foundation for understanding PID control theory and the transfer functions required for inverted pendulum stability.
2. Åström, K. J., & Hägglund, T. (2006). *Advanced PID Control*. ISA - The Instrumentation, Systems, and Automation Society. This reference was instrumental in designing the anti-windup logic and derivative filtering implemented in the firmware.
3. InvenSense Inc. (2013). *MPU-6000 and MPU-6050 Product Specification Revision 3.4*. This technical datasheet provided the register map, I²C communication protocols, and scaling factors essential for accurate sensor data acquisition.
4. Atmel Corporation. (2015). *ATmega2560 8-bit AVR Microcontroller Datasheet*. This documentation provided the technical specifications regarding timer interrupts, I/O pin current limits, and power management requirements for the Arduino Mega processing unit.
5. Arduino Documentation Team. (2024). *Arduino Wire Library Reference*. Retrieved from the official Arduino website (www.arduino.cc). This resource facilitated the implementation of the I²C communication interface used to poll the MPU6050 sensor.
6. Monaghan, M. (2019). *AccelStepper Library Documentation*. Retrieved from the official AccelStepper documentation (www.airspayce.com). This library was used as the framework for non-blocking motor pulse generation and step/direction management.
7. Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2014). *Feedback Control of Dynamic Systems* (7th ed.). Pearson. This comprehensive source offered insights into the discrete-time implementation of control loops, which was directly applied to the 100Hz timing gate logic used in the firmware.